



PUESTA EN PRODUCCIÓN
SEGURA



PRÁCTICA 4C CONFIGURACIÓN SEGURA DE TLS



Carles Ochoa



Índice

Introducción.....	3
¿Qué es TLS (Activdad 1)?.....	3
¿Qué es AES (Activdad 2)?.....	3
Actividad 1: Configuración Segura de TLS.....	3
Actividad 2: Cifrado de Datos Sensibles con AES.....	10

Introducción

En esta actividad vamos a trabajar la protección de la información tanto en tránsito como en reposo. Para ello, primero vamos a configurar TLS 1.3 en un servidor web con el fin de asegurar las comunicaciones entre cliente y servidor. Después, aplicaremos cifrado AES para proteger datos sensibles almacenados, comparando un ejemplo inseguro con uno seguro. El objetivo es comprender y aplicar medidas básicas pero efectivas de seguridad en sistemas y aplicaciones web.

¿Qué es TLS (Activdad 1)?

TLS (Transport Layer Security) es un protocolo de seguridad que cifra la comunicación entre un cliente y un servidor, evitando que terceros puedan interceptar o modificar los datos mientras viajan por la red.

¿Qué es AES (Activdad 2)?

AES (Advanced Encryption Standard) es un algoritmo de cifrado simétrico utilizado para proteger datos sensibles almacenados. Permite que la información solo pueda ser leída por quien tenga la clave correcta, garantizando confidencialidad y seguridad.

Actividad 1: Configuración Segura de TLS

Instalación de OpenSSL y generación de un certificado TLS

En este primer paso instalaremos OpenSSL, una herramienta necesaria para crear certificados digitales, y se genera un certificado TLS autofirmado que permitirá habilitar conexiones seguras HTTPS en el servidor web.

Los comandos a ejecutar son:

- `sudo apt update`
- `sudo apt install openssl`

Una vez instalado openssl, crearemos un directorio para almacenar los certificados y generaremos uno válido para un año.

```
(carles@kalicarles)-[~]
$ sudo mkdir -p /etc/apache2/ssl

(carles@kalicarles)-[~]
$ sudo openssl req -newkey rsa:2048 -nodes \
-keyout /etc/apache2/ssl/server.key \
-x509 -days 365 \
-out /etc/apache2/ssl/server.crt \
-subj "/C=US/ST=State/L=City/O=Organization/OU=Department/CN=localhost" \
-addext "basicConstraints=CA:FALSE" \
-addext "subjectAltName=DNS:localhost"
.....+.....+.....+...+..+.....+
.....+...+..+...+.....+...+...+.....+.....+.....+.....+.....+.....+.....+.....+
.....+...+...+.....+...+..+.....+..+.....+...+.....+.....+.....+.....+.....+
..+.....+.....+.....+.....+.....+.....+.....+.....+.....+.....+.....+.....+.....+
.....+...+...+.....+.....+.....+.....+.....+.....+.....+.....+.....+.....+.....+
_____

(carles@kalicarles)-[~]
$ ls /etc/apache2/ssl
server.crt  server.key

(carles@kalicarles)-[~]
```

Configurar Apache para usar TLS

En este paso configuraremos el servidor Apache para que utilice el certificado TLS generado previamente y permita conexiones seguras mediante HTTPS.

Primero, editamos el archivo de configuración SSL por defecto de Apache:

```
sudo nano /etc/apache2/sites-available/default-ssl.conf
```

Dentro del archivo configuraremos el bloque VirtualHost para el puerto 443, indicando la ubicación del certificado y la clave privada:

```
Session Acciones Editar Vista Ayuda
GNU nano 8.6
<VirtualHost *:443>
  ServerAdmin admin@localhost
  ServerName localhost
  DocumentRoot /var/www/html

  SSLEngine on
  SSLCertificateFile /etc/apache2/ssl/server.crt
  SSLCertificateKeyFile /etc/apache2/ssl/server.key

  SSLProtocol TLSv1.2 TLSv1.3
  SSLCipherSuite HIGH:!aNULL:!MD5
</VirtualHost>
```

Con esta configuración:

- Se habilita el uso de SSL/TLS en Apache.
- Se cargan el certificado y la clave privada.
- Se deshabilitan versiones antiguas e inseguras de TLS.
- Se permiten únicamente cifrados seguros.

Habilitar SSL y reiniciar Apache

Una vez configurado Apache para usar TLS, es necesario habilitar el módulo SSL y el sitio seguro para que la configuración entre en funcionamiento.

Primero habilitamos el módulo SSL y el sitio SSL por defecto y después reiniciamos el servicio Apache para aplicar todos los cambios realizados.

```
(carles@kalicarles)-[~]
$ sudo a2enmod ssl

Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
    systemctl restart apache2

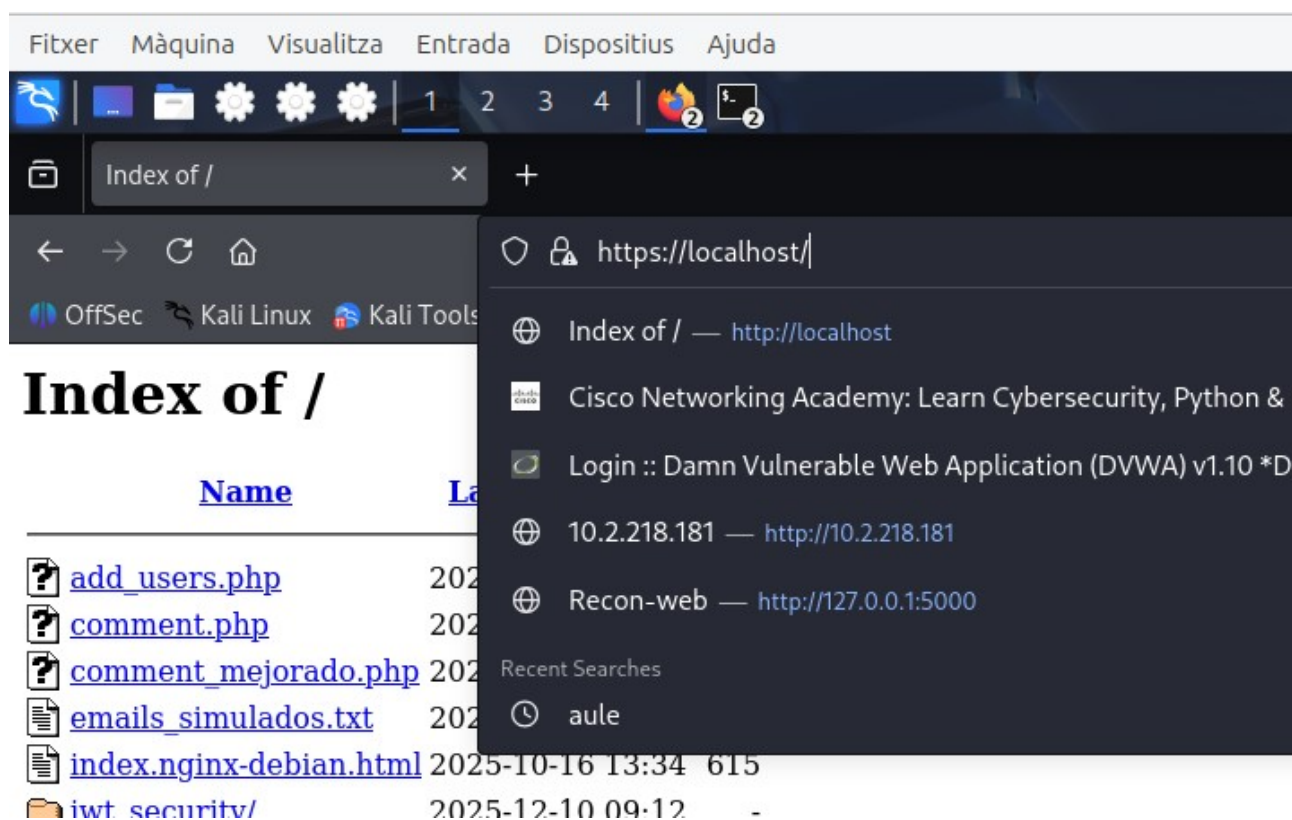
(carles@kalicarles)-[~]
$ sudo a2ensite default-ssl

Enabling site default-ssl.
To activate the new configuration, you need to run:
    systemctl reload apache2

(carles@kalicarles)-[~]
$ sudo systemctl restart apache2

(carles@kalicarles)-[~]
$
```

Y en el navegador introduciendo <https://localhost> podemos comprobar si la configuración ha funcionado. En este caso es correcto.



Mitigación de problemas comunes

Para mejorar la seguridad y evitar el uso de versiones antiguas de TLS, es recomendable deshabilitar protocolos inseguros. En el archivo default-ssl.conf se configura:

```
SSLProtocol TLSv1.2 TLSv1.3
```

Además, se puede forzar el uso de HTTPS mediante HSTS, añadiendo dentro del bloque <VirtualHost *:443>:

```
Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"
```

Por lo tanto editamos el archivo:

```
sudo nano /etc/apache2/sites-available/default-ssl.conf para introducir la linea.
```

```

GNU nano 8.6
<VirtualHost *:443>
Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"
ServerAdmin admin@localhost
ServerName localhost
DocumentRoot /var/www/html

SSLEngine on
SSLCertificateFile /etc/apache2/ssl/server.crt
SSLCertificateKeyFile /etc/apache2/ssl/server.key

SSLProtocol TLSv1.2 TLSv1.3
SSLCipherSuite HIGH:!aNULL:!MD5
</VirtualHost>

```

Y habilitar los encabezados para que esta directiva funcione.

```
sudo a2enmod headers
```

```
sudo systemctl restart apache2
```

¿Cómo eliminar la advertencia del candado?

1. Crear un Certificado de Autoridad (CA)

Para eliminar la advertencia de seguridad del navegador causada por el certificado autofirmado, crearemos una Autoridad Certificadora (CA) local. Esta CA se utilizará para firmar el certificado del servidor, de modo que el navegador pueda confiar en él una vez importada.

Primero creamos el directorio para almacenar los certificados y accedemos a él

```
sudo mkdir -p /etc/apache2/ssl
```

```
cd /etc/apache2/ssl
```

Una vez dentro del directorio generamos la clave privada de la CA y el certificado raíz de la CA, con una validez de 10 años.

```

(carles@kalicarles)-[/etc/apache2/ssl]
$ sudo openssl genrsa -out myCA.key 2048

(carles@kalicarles)-[/etc/apache2/ssl]
$ sudo openssl req -x509 -new -nodes -key myCA.key -sha256 -days 3650 -out myCA.pem \
-subj "/C=US/ST=State/L=City/O=MyCompany/OU=IT Department/CN=My Local CA"

(carles@kalicarles)-[/etc/apache2/ssl]
$

```

Con estos pasos obtenemos:

- **myCA.key**: clave privada de la Autoridad Certificadora.
- **myCA.key**: clave privada de la Autoridad Certificadora.

Este certificado será el que se importe posteriormente en el navegador para que los certificados firmados por esta CA sean reconocidos como seguros.

2. Firmar el certificado SSL con la CA local

En este paso generaremos un nuevo certificado para el servidor Apache, que será firmado por la Autoridad Certificadora (CA) local creada anteriormente. Esto permite que el navegador confíe en el certificado y elimine la advertencia del candado una vez importada la CA.

Primero generamos la clave privada del servidor:

```
(carles@kalicarles)-[/etc/apache2/ssl]
$ sudo openssl genrsa -out server.key 2048

(carles@kalicarles)-[/etc/apache2/ssl]
$
```

Después, creamos una solicitud de firma de certificado (CSR), que contiene la información del servidor:

```
(carles@kalicarles)-[/etc/apache2/ssl]
$ sudo openssl req -new -key server.key -out server.csr \
-subj "/C=US/ST=State/L=City/O=MyCompany/OU=IT Department/CN=localhost"

(carles@kalicarles)-[/etc/apache2/ssl]
$
```

A continuación, firmamos el certificado del servidor utilizando la CA local:

```
(carles@kalicarles)-[/etc/apache2/ssl]
$ sudo openssl x509 -req -in server.csr -CA myCA.pem -CAkey myCA.key \
-CACreateserial -out server.crt -days 365 -sha256

Certificate request self-signature ok
subject=C=US, ST=State, L=City, O=MyCompany, OU=IT Department, CN=localhost

(carles@kalicarles)-[/etc/apache2/ssl]
$
```

Con esto se generan los siguientes archivos:

- **server.key**: clave privada del servidor.
- **server.crt**: certificado del servidor firmado por la CA local.
- **myCA.srl**: archivo de control de la CA.

Este certificado firmado sustituirá al certificado autofirmado anterior y permitirá una conexión HTTPS sin advertencias.

3. Configurar Apache con el Nuevo Certificado

Una vez generado el certificado del servidor firmado por la **CA local**, es necesario configurar Apache para que utilice estos nuevos archivos y así eliminar la advertencia de seguridad del navegador.

Primero, editamos el archivo de configuración SSL de Apache:


```
sudo nano /etc/apache2/sites-available/default-ssl.conf
```

Y cambiaremos las rutas para que apunten a los nuevos archivos:

```
SSLEngine on
```

```
SSLCertificateFile /etc/apache2/ssl/server.crt
```

```
SSLCertificateKeyFile /etc/apache2/ssl/server.key
```

```
SSLCertificateChainFile /etc/apache2/ssl/myCA.pem
```

```

Session Acciones Editar Vista Ayuda
GNU nano 8.6
<VirtualHost *:443>
Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"
ServerAdmin admin@localhost
ServerName localhost
DocumentRoot /var/www/html

SSLEngine on
SSLCertificateFile /etc/apache2/ssl/server.crt
SSLCertificateKeyFile /etc/apache2/ssl/server.key
SSLCertificateChainFile /etc/apache2/ssl/myCA.pem

SSLProtocol TLSv1.2 TLSv1.3
SSLCipherSuite HIGH:!aNULL:!MD5
</VirtualHost>

```

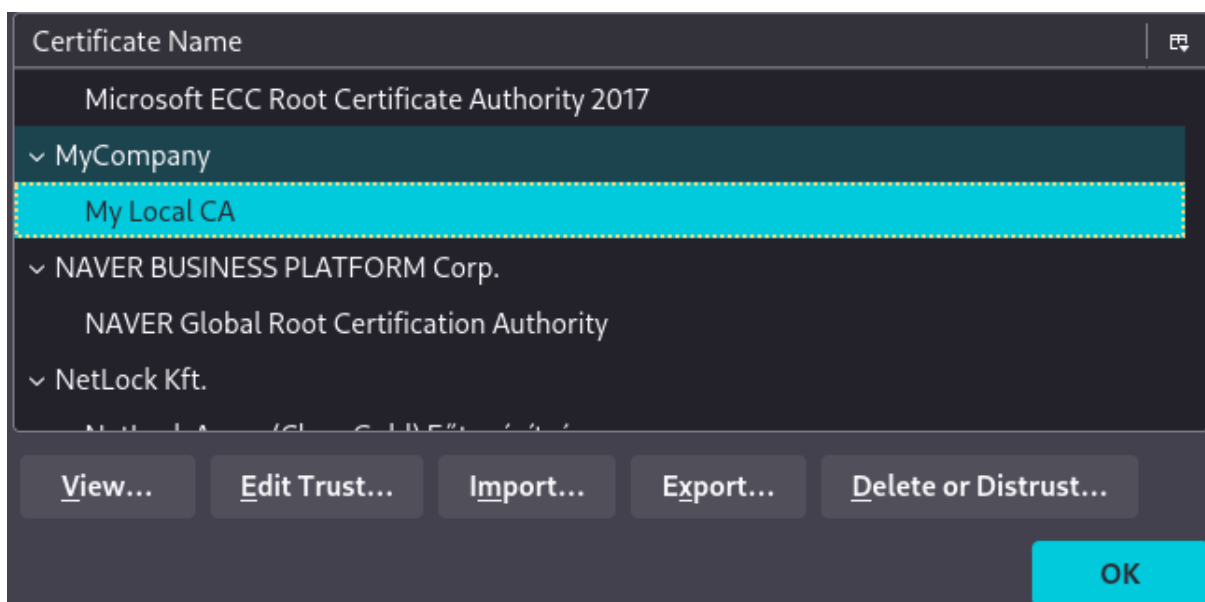
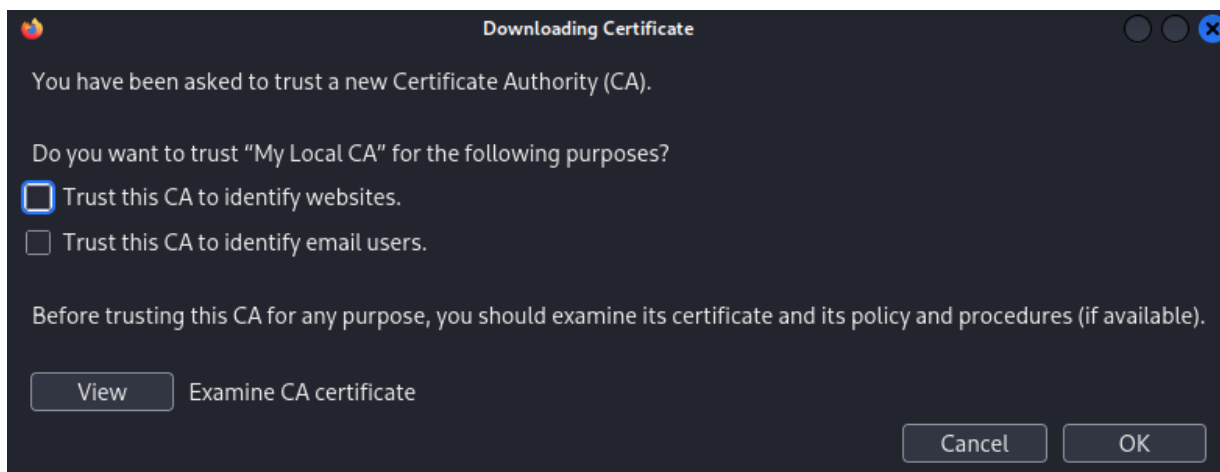
Después de guardar los cambios, reiniciamos Apache para aplicar la nueva configuración

```
sudo systemctl restart apache2
```

Importar la CA en Firefox

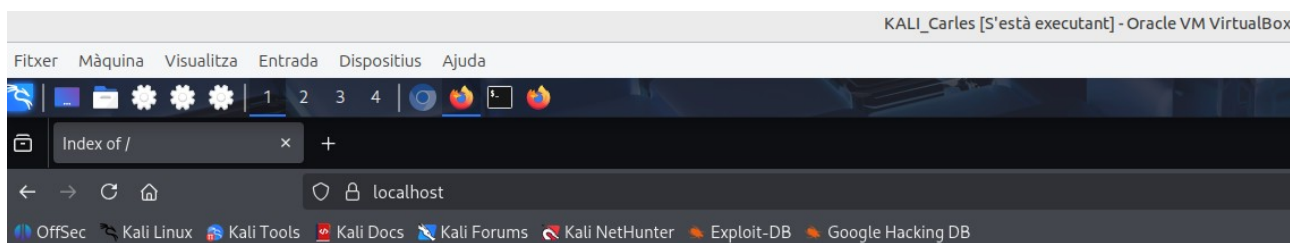
Por último nos faltará importar el certificado en el firefox. Para ello seguiremos los siguientes pasos:

1. Abrir Firefox e ir a `about:preferences#privacy`.
2. En Certificados y seleccionar Ver certificados.
3. En la pestaña Autoridades y seleccionar Importar y seleccionamos el `/etc/apache2/ssl/myCA.pem`.
4. Marcamos la casilla de Confiar en esta CA para identificar sitios web.
5. Guardamos los cambios.



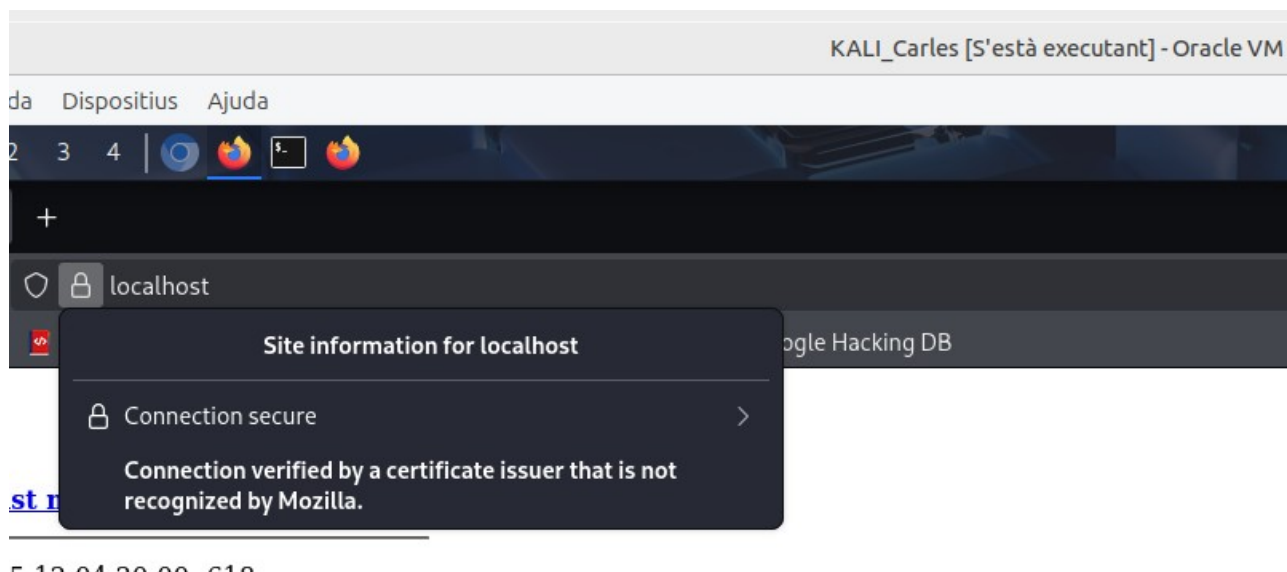
Si ahora introducimos la url <https://localhost> veremos:

- Que el candado está cerrado (sin advertencias).
- Connection secure



Index of /

[Name](#) [Last modified](#) [Size](#) [Description](#)



Por lo tanto podemos confirmar que ahora sí el navegador confía en el certificado SSL/TLS del servidor local porque hemos importado y marcado como confiable la Autoridad Certificadora (CA) local que firmó dicho certificado, eliminando así las advertencias de seguridad y estableciendo una conexión HTTPS válida y segura.

Actividad 2: Cifrado de Datos Sensibles con AES

En esta segunda actividad trabajaremos la protección de datos en reposo, es decir, los datos que se almacenan en el servidor. Para ello utilizaremos el algoritmo de cifrado AES, comparando un ejemplo inseguro (sin cifrado) con uno seguro, donde la información se cifra antes de guardarse y se descifra al recuperarse.

El objetivo es comprender por qué almacenar datos en texto plano es inseguro y cómo AES permite proteger información sensible incluso si alguien accede al servidor.

Ejemplo inseguro: API sin cifrado (texto plano)

En este apartado veremos un ejemplo de API insegura, donde los datos se envían y se almacenan en texto plano, sin ningún tipo de cifrado. Esto supone un riesgo grave, ya que la información puede ser interceptada o leída por terceros.

Para ello crearemos un nuevo directorio de trabajo.

```
(carles@kalicarles)-[~]
$ sudo mkdir -p /var/www/html/aes

[sudo] contraseña para carles:

(carles@kalicarles)-[~]
$ sudo chown -R www-data:www-data /var/www/html/aes

(carles@kalicarles)-[~]
$ sudo chmod 755 /var/www/html/aes

(carles@kalicarles)-[~]
```

Dentro de este nuevo directorio, crearemos el código inseguro.

```
(carles@kalicarles)-[~]
$ cat /var/www/html/aes/insecure_api.php
<?php
header("Content-Type: application/json");

// Simulación de almacenamiento en un archivo
$file = "data.txt";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $data = json_decode(file_get_contents("php://input"), true);

    if (!isset($data["text"])) {
        echo json_encode(["error" => "Texto vacío"]);
        exit;
    }

    // Guardar el mensaje en texto plano (INSEGURO)
    file_put_contents($file, $data["text"]);
    echo json_encode(["message" => "Texto guardado"]);
}

if ($_SERVER["REQUEST_METHOD"] === "GET") {
    if (!file_exists($file)) {
        echo json_encode(["error" => "No hay mensajes"]);
        exit;
    }

    // Devolver el mensaje en texto plano (INSEGURO)
    $message = file_get_contents($file);
    echo json_encode(["plaintext" => $message]);
}
?>
```

¿Por qué es inseguro?

- **Datos sin cifrar:** la información viaja y se almacena en texto plano.
- **Fácilmente interceptable:** un atacante puede leer los mensajes.
- **Sin protección:** cualquiera con acceso al servidor puede ver o modificar los datos.

Para comprobar el funcionamiento, enviaremos un mensaje a la API mediante una petición POST y posteriormente lo recuperaremos con una petición GET, observando que los datos se almacenan y transmiten en texto plano, lo que representa un riesgo de seguridad.

Creamos el fichero data.txt con los permisos adecuados:

```
(carles@kalicarles)-[/var/www/html/aes]
$ sudo touch data.txt

(carles@kalicarles)-[/var/www/html/aes]
$ sudo chown www-data:www-data /var/www/html/aes/data.txt

(carles@kalicarles)-[/var/www/html/aes]
$ sudo chmod 664 /var/www/html/aes/data.txt

(carles@kalicarles)-[/var/www/html/aes]
$
```

Ahora usamos curl para probar la API insegura y comprobar que los datos viajan en texto plano.

Primeramente enviamos un texto (POST) sin cifrar a la API:

```
(carles@kalicarles)-[/var/www/html/aes]
$ curl -k -X POST https://localhost/aes/insecure_api.php \
-H "Content-Type: application/json" \
-d '{"text":"Contra:Carles123"}'

{"message":"Texto guardado"}
```

Ahora leemos el mensaje guardado (GET), también en texto plano:

```
(carles@kalicarles)-[/var/www/html/aes]
$ curl -k -X GET https://localhost/aes/insecure_api.php

{"plaintext":"Contra:Carles123"}

(carles@kalicarles)-[/var/www/html/aes]
$
```

Y como podemos observar el mensaje se envía y se almacena sin cifrar por lo tanto esto es completamente inseguro porque cualquiera que intercepte la comunicación puede leerlo.

Ejemplo seguro: API con AES-GCM

Ahora mediante un código totalmente seguro vamos a probar la API segura, que cifra los datos antes de guardarlos usando AES.

Para ello en el mismo directorio, crearemos el archivo seguro.txt con los permisos correspondientes.

```
(carles@kalicarles)-[/var/www/html/aes]
$ sudo touch seguro.txt

(carles@kalicarles)-[/var/www/html/aes]
$ sudo chown www-data:www-data seguro.txt

(carles@kalicarles)-[/var/www/html/aes]
$ sudo chmod 664 seguro.txt

(carles@kalicarles)-[/var/www/html/aes]
$
```

Una vez realizado este paso, crearemos los códigos para el correcto funcionamiento del cifrado.

Primero crearemos el archivo config.php para definir la clave secreta que se usará para cifrar y descifrar los datos.

```
(carles@kalicarles)-[/var/www/html/aes]
$ cat config.php
<?php
define("SECRET_KEY", hex2bin(
"00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
));
?>

(carles@kalicarles)-[/var/www/html/aes]
$
```

El siguiente archivo a crear es secure_api.php para cifrar los datos antes de guardarlos con AES-256-GCM que es seguro y autenticado.

```
(carles@kalicarles)-[/var/www/html/aes]
$ cat secure_api.php
<?php
header("Content-Type: application/json");
include "config.php";

$secretKey = SECRET_KEY;
$file = "data_secure.txt";

function encryptData($plaintext, $key) {
    $iv = random_bytes(12);
    $ciphertext = openssl_encrypt(
        $plaintext,
        "aes-256-gcm",
        $key,
        OPENSSL_RAW_DATA,
        $iv,
        $tag
    );
    return base64_encode($iv . $tag . $ciphertext);
}

function decryptData($ciphertext, $key) {
    $data = base64_decode($ciphertext);
    $iv = substr($data, 0, 12);
    $tag = substr($data, 12, 16);
    $ciphertext = substr($data, 28);
    return openssl_decrypt(
        $ciphertext,
        "aes-256-gcm",
        $key,
        OPENSSL_RAW_DATA,
        $iv,
        $tag
    );
}
```

Por lo tanto con todos los php ya creados realizaremos las pruebas.

Enviamos un mensaje en texto plano a la API (POST).

```
(carles@kalicarles)-[/var/www/html/aes]
$ curl -k -X POST https://localhost/aes/secure_api.php \
-H "Content-Type: application/json" \
-d '{"text": "Password:Carles123"}'
{"message": "Texto cifrado y almacenado"}

(carles@kalicarles)-[/var/www/html/aes]
$
```

Y ahora mediante GET recuperamos el mensaje encriptado y luego descifrado.

```
(carles@kalicarles)-[/var/www/html/aes]
$ curl -k -X GET https://localhost/aes/secure_api.php
{"plaintext": "Password:Carles123"}

(carles@kalicarles)-[/var/www/html/aes]
$
```

Ahora sí que nuestro código cumple con los principios de seguridad, ya que los datos se transmiten mediante HTTPS y además se almacenan cifrados con AES. De este modo, aunque un atacante intercepte la comunicación o acceda al servidor, no podrá leer la información sin la clave de cifrado.